



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 5

Introduction to Design Patterns

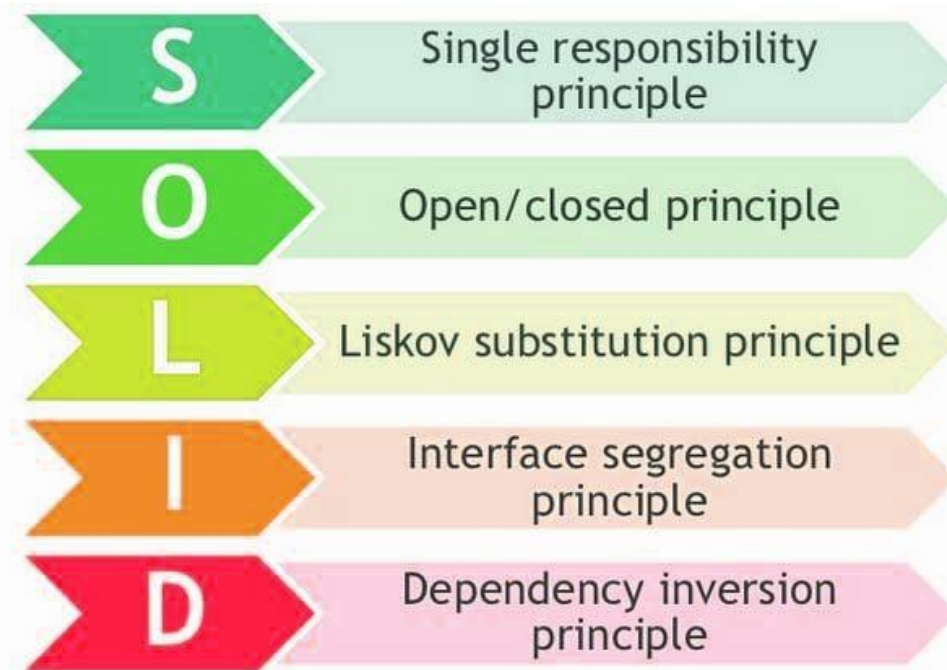


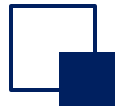
Chapter 5: Introduction to Design Patterns (Part I)

- **History**
- **Object-oriented design principles**
- **Design patterns**

Object-oriented design principles

- There are common principles to follow when designing object-oriented programs, including:





Single responsibility principle

□ **The Single Responsibility Principle (SRP) defines:**

□ **For a class, there should be only one reason for its change.**

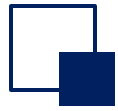
- Don't group different responsibilities, as changes can affect unrelated responsibilities.
- If a class has too many responsibilities, the less likely it is to be reused, the more likely it is to couple those responsibilities together, and a change in one responsibility may weaken and inhibit the class's ability to accomplish other duties.
- If there is more than one motivation to change a class, then the class has more than one responsibility, and the separation of duties of the class should be considered.



Open-closed principle

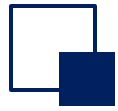
▣ Open-Closed Principle – OCP

- ▣ **A software entity should be open to extensions and closed to modifications. That is, when designing a module, the module should be extended without modification.**
 - By extending existing software systems, new behaviors can be provided to meet new requirements for software, so that changing software systems have a certain degree of adaptability and flexibility.
 - In object-oriented programming, through abstract classes and interfaces, the features of concrete classes are specified as abstraction layers, which are relatively stable and do not need to be changed, so as to meet the requirements of "closed to modifications ". Concrete classes derived from abstract classes can change the behavior of the system, thus satisfying the "open to extensions".



Liskvo Substitution Principle

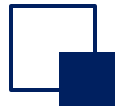
- ❑ **Definition of the Liskvo Substitution Principle (LSP):**
- ❑ Objects of a subclass should be replaceable with objects of the superclass without breaking the program's behavior.
- ❑ The reverse substitution does not work, and if a software entity uses a subclass, then it does not necessarily apply to the base class.



Interface Segregation Principle

□ Interface Segregation Principle – ISP: The client should not depend on interfaces it does not need.

- It is better to use multiple specialized interfaces than to use a single master interface. That is, the dependencies of one class on another class should be built on the smallest interface.
- Once an interface is too large, it needs to be split into smaller interfaces, and the client using the interface only needs to know the methods associated with it.



Dependency Inversion Principle

- ▣ **Definition of Dependency Inversion Principle (DIP):**
- ▣ **High-level modules shouldn't depend on low-level modules, they should all rely on abstractions. Program should focus on the interface, not the implementation.**
 - Instead of concrete classes, you should use interfaces and abstract classes for variable type declarations, parameter type declarations, method return type descriptions, and data type transformations.
 - The traditional design approach to process systems tends to make high-level modules dependent on low-level modules and abstract levels dependent on concrete levels. The inversion principle is to turn this false dependency upside down.



Composite/Aggregate Reuse Principle

- ▣ **Definition of the Composite/Aggregate Reuse Principle (CARP): Maximize the use of object composition, rather than inheritance for the purpose of reuse.**
 - Use some existing objects inside a new object to make them part of the new object; New objects are delegated to reuse these objects.
 - Composite/aggregation should be used first, and inheritance should be considered second. When using inheritance, the principle of LSP should be strictly followed. Effective use of inheritance will help to understand the problem and reduce complexity, while misuse of inheritance will increase the difficulty and complexity of the system in software construction and maintaining.



Law of Demeter

▣ Law of Demeter – LoD:

A software entity should interact with other entities as little as possible.

- Each software unit has minimal knowledge of the others and is limited to those software units that are closely related to the unit.
- In this way, when one module is modified, it will affect the other modules as little as possible. Scaling will be relatively easy. This is a restriction on communication between software entities. It requires limiting the width and depth of communication between software entities.



Chapter 5: Introduction to Design Patterns (Part I)

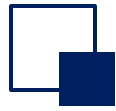
- History
- Object-oriented design principles
- Design patterns

The concept of design patterns

Design patterns

It is a summary of the experience of code design that has been used repeatedly and cataloged by classification. It describes some of the recurring problems in the software design process, and the core solutions to those problems. The purpose is to improve the reusability of the code, the readability of the code, and the reliability of the code.

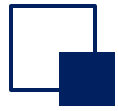
The essence of design patterns is the practical application of object-oriented design principles, and a full understanding of the encapsulation, inheritance, and polymorphism of classes, as well as the relational and combinatorial relationships of classes.



The concept of design patterns

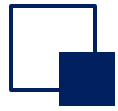
The four essential elements of a design pattern

element	description
Pattern Name	A mnemonic phrase that uses one or two words to describe the problem, solution, and effect of the pattern.
Problem	Describes the application environment of the pattern, i.e., when to use it. It explains the design problem and the causes and consequences of its existence, as well as a set of prerequisites that must be met.
Solution	The components of the design, their interrelationships, and their respective responsibilities and ways of working together are described. The solution does not describe a specific and specific design or implementation, but rather provides an abstract description of the design problem and how to solve it with a general combination of elements (a combination of classes or objects).
Consequence	It describes the effects of the application of the pattern and the trade-offs that should be made when using the pattern, i.e., the advantages and disadvantages of the pattern. It is mainly the measurement of time and space, as well as the impact of the model on the flexibility, scalability, and portability of the system.



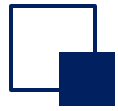
The benefits of design patterns

- Reuse design: Reusing design makes more sense than reusing code, and it automatically leads to code reuse.
- To provide a common vocabulary for design, each pattern name is a design vocabulary, and the concept is that it is easier for programmers to communicate with each other.
- Adopting a schema vocabulary in your development documentation makes it easier for others to understand your ideas and practices, and makes it easier to write development documentation.
- Applying design patterns makes it easy to refactor the system, ensures that the right code is developed, and reduces the likelihood of errors in the design or implementation.
- Support for variation, which can provide a good system architecture for rewriting other applications.
- When used correctly, you can save a lot of time.



The categorization of design patterns

- Categorized by purpose (what the pattern is used to accomplish)
 - **Creational patterns** : Used to describe "how to create objects", its main feature is "separating the creation and use of objects".
 - **Structural patterns** : Used to describe how a class or object can be arranged into larger structures.
 - **Behavioral patterns**: Used to describe how classes or objects work together to accomplish tasks that no single object can do alone, and how responsibilities are assigned.
- Categorized by scope (patterns are mainly used for classes or objects)
 - **Class patterns** : Used to handle relationships between classes and subclasses, which are established through inheritance, are static, and are determined at compile time.
 - **Object patterns** : Used to deal with relationships between objects, which can be achieved by combining or aggregating, and can be changed at the runtime, making them more dynamic.



Classification of design patterns

		Purpose		
		Creational	Structural	Behavioral
Scope	class	Factory Method	Adapters (Classes)	Template method, Interpreter
	object	Abstract factory Builder Prototype Singleton	Adapter (Object) Proxy Composite Decorator Flyweight Facade Bridge	Chain of Responsibility Iterator Strategy Mediator Observer Command State Memento Visitor



Adapters

- ❑ The Adapter pattern allows incompatible interfaces to work together by acting as a bridge that converts one interface into another that clients expect.
- ❑ Advantages
 - Enables reusability of existing classes
 - allows integration of legacy systems without modifying their source code



Decorator

- ❑ The Decorator pattern dynamically adds new responsibilities to an object without altering its structure, by wrapping it with additional behavior.
- ❑ Advantages
 - Provides flexible alternative to subclassing for extending functionality
 - Follows open-closed principle
 - Allows mix-and-match of features at runtime



Command

- ❑ Command Pattern turns requests into objects that can be stored, queued, and undone.
- ❑ Advantages
 - Undo/Redo - Commands can remember state for reversal
 - Queueing - Commands can be stored and executed later
 - Decoupling - The invoker doesn't need to know request details
 - Extensible - Easy to add new commands without changing existing code



Singleton pattern

- **Singleton pattern**

Ensure that there is only one instance of a class and provide a global access point to access it.

- Many times we have to make sure that there is only one instance of a class
 - There is only one instance of connection in the entire application that connects to the database
 - The operating system has only one clock
 - There is only one hero machine in the plane war game...



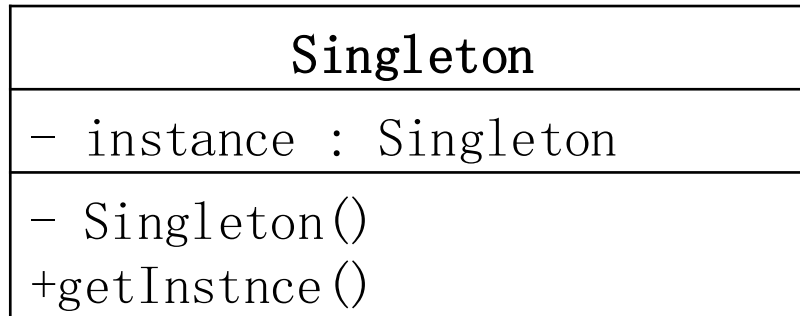
Singleton pattern



How do you ensure that there is only one instance of a class, and that this instance is easily accessible?

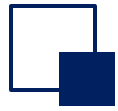
A global variable makes an object globally accessible, but it doesn't prevent you from instantiating multiple objects. One of the best ways to do this is to have the class itself responsible for saving its unique instance, which guarantees that no other instances can be created, and that it can provide a way to access that instance.

Singleton schema diagram



- - -

Singleton class, which defines a getInstance() operation that allows customers to access a unique instance of it. getInstance() is a static method that is primarily responsible for creating its own unique instance.



Singleton pattern (1): Eager initialization

Implemented in the classic singleton pattern

The eager initialization is instantiated immediately when the class is loaded, and only one instance will appear when it is used in the future.

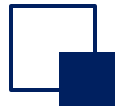
Singleton class, which defines a getInstance() operation that allows customers to access a unique instance of it.

```
public class Singleton{  
    private static Singleton singleton = new Singleton ();  
  
    private Singleton(){  
    }  
  
    public static Singleton GetInstance () {  
        return singleton;  
    }  
}
```

1. When the class is initialized, the object is loaded immediately, which is a natural linear security

2. Declare the constructor as private so that the outside world cannot use new to create such instances

3. Methods do not need to be synchronized, and the call efficiency is high



Singleton pattern (2): Lazy initialization

Implemented in the classic singleton pattern

Instead of instantiating the class when it is loaded, it is instantiated when the specified instance method is called, so that it is not instantiated when you don't want to use it. Generally more efficient than the Eager initialization.

Singleton class, which defines a getInstance() operation that allows customers to access a unique instance of it. getInstance() is a static method that is primarily responsible for creating its own unique instance.

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton (){}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

1. Use a private static variable to record a unique instance of the Singleton class

2. Declare the constructor as private so that the outside world cannot use new to create such instances

3. Use the public static method getInstance() to instantiate the object and return that unique instance

Singleton Pattern (3): Thread-Safe Lazy Initialization



If multiple threads access the Singleton class at the same time, call the `getInstance()` method, it is possible to create multiple instances, how to avoid it?

By adding the `synchronized` keyword to the `getInstance()` method, each thread is forced to wait for other threads to leave the method before entering the method. That is, no two threads can enter the method at the same time.

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton (){}  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

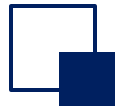
Thread-safe, but inefficient!

Extracurricular development: double-checked locking



Singleton pattern

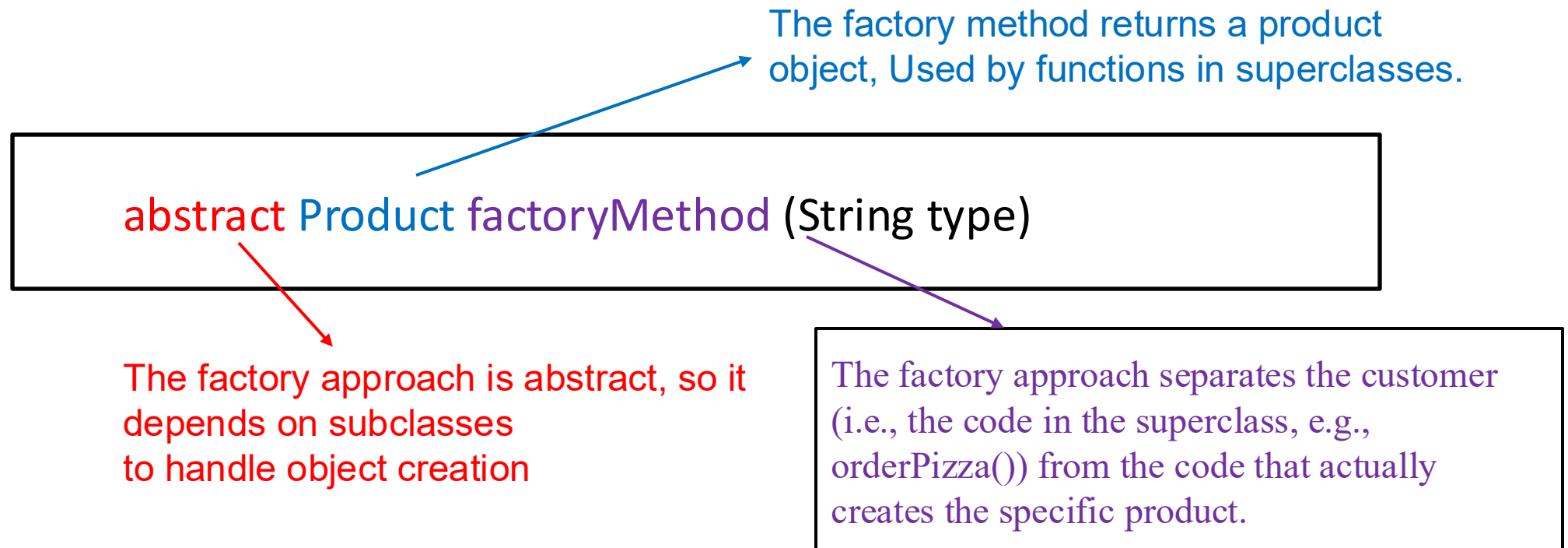
element	description
Pattern Name	Singleton Pattern
Intent	You just want to have an object that can be accessed from all parts of the system, without using global variables and without passing references to objects.
Problem	How can I ensure that several different customer objects want to reference the same object?
Solution	A unique instance is guaranteed 1) Define a private static member variable to save a reference to a single instance; 2) define a public static method getInstance() to get a unique instance; 3) The class is responsible for instantiating the object "on first use".
Consequence	Advantages: 1) Controlled access to unique instances; 2) There is only one instance in memory, which reduces the memory overhead, avoids frequent creation and destruction of objects, and improves performance. 3) avoid multiple occupation of shared resources; 4) allow a variable number of instances; 5) Can be accessed globally.



Factory Method

Factory Method

It is used to handle the creation of objects and encapsulate such behavior in subclasses. In this way, the code in the client program about superclasses is decoupled from the code for creating subclass objects.



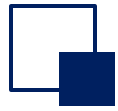


Simple Factory Mode VS Factory Mode

- **Simple factory**
 - The logical judgment of creating an object is placed in the factory class, and the customer does not perceive the specific class, but it violates the open-closed principle, and if you want to add a new pizza type, you have to modify the factory class.
- **Factory method**
 - If a specific class is added through extension, it is in line with the principle of open and closed, but the specific factory class must be perceived on the client side, that is, the judgment logic must be moved from the factory class of a simple factory to the client.
 - If there is a franchise store in a new region, you only need to create the corresponding pizzeria subclass and pizza subclass to implement the abstract factory interface and abstract product interface, without modifying the existing code.

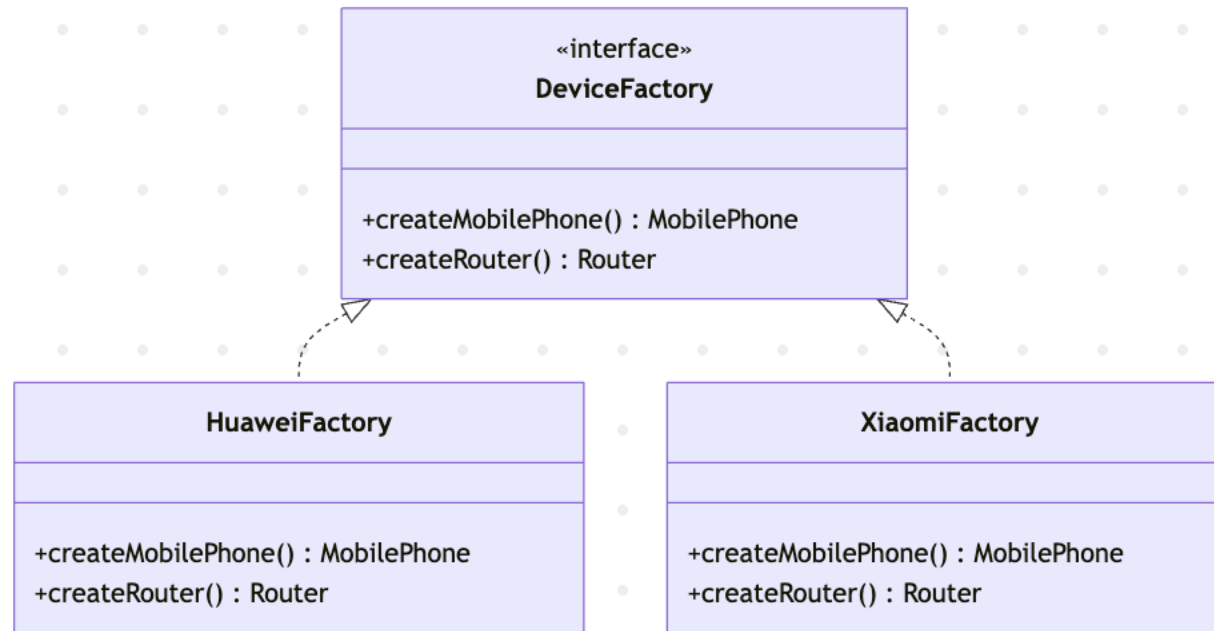
An overview of the Abstract Factory pattern

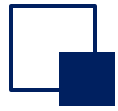
- The Abstract Factory Pattern is the creation of other factories around one gigafactory. This gigafactory is also known as a factory for other factories. This type of design pattern is a creative pattern that provides a way to create objects.



Abstract factory pattern

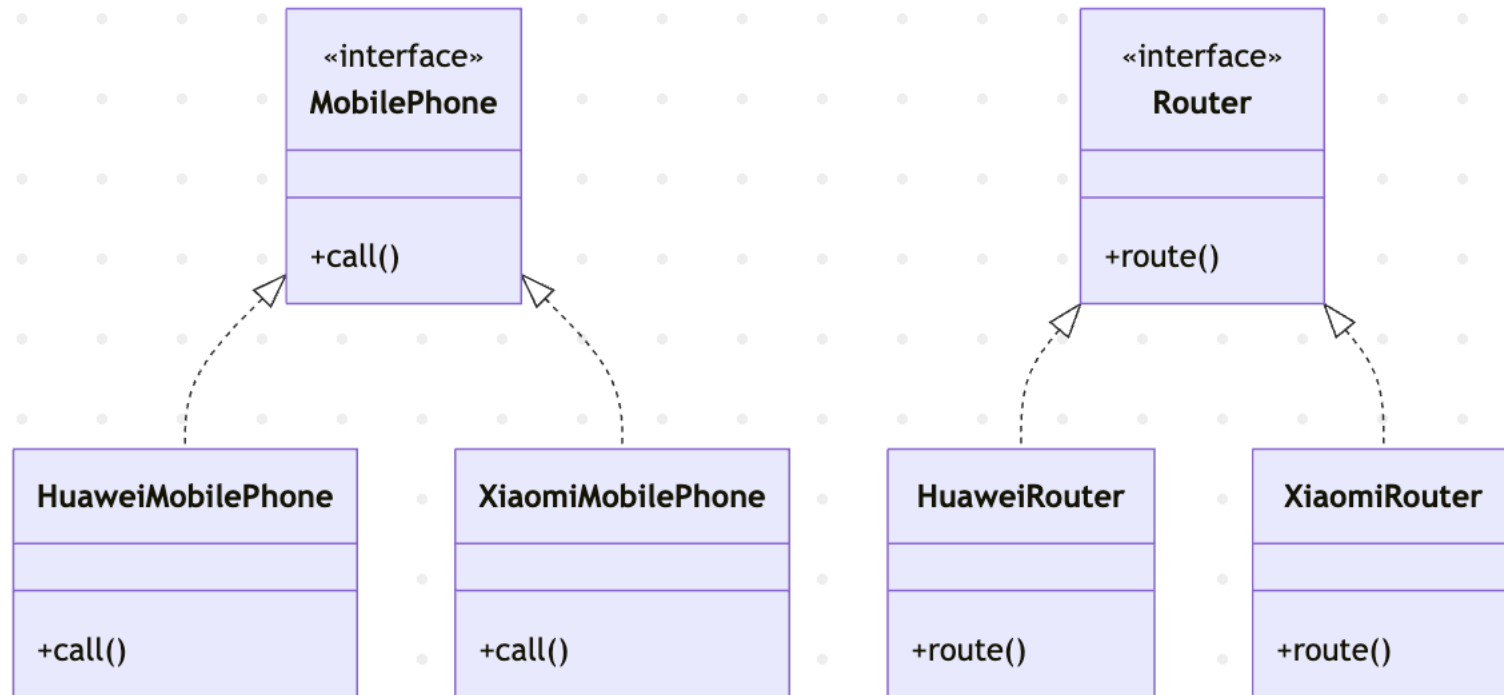
Class Diagram Analysis:





Abstract factory pattern

Class Diagram Analysis:





Abstract factory pattern (expand a product)

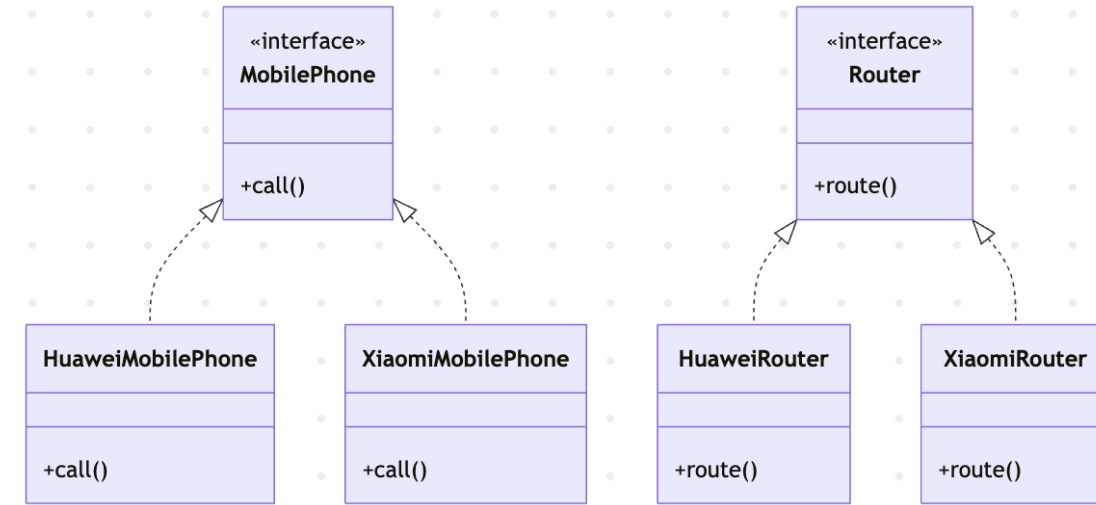
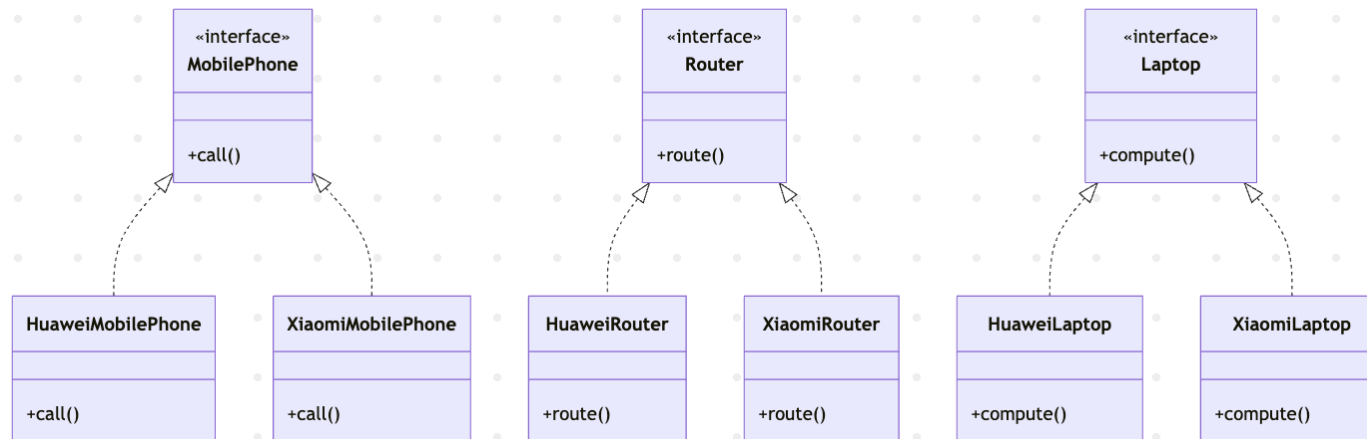
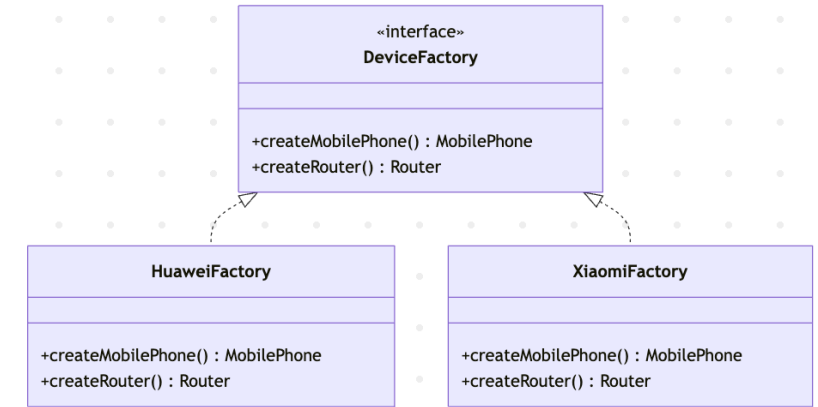
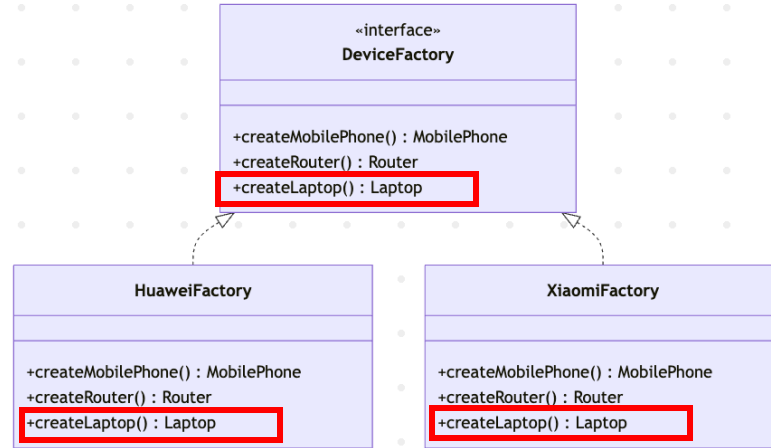
Now looking back at the concept of product family and product level, what happens if you add a new brand product family or product type?

Expand a product type

➤ We will find that it is very difficult to expand a product type, for example, adding laptops to the product type, which means that Huawei and Xiaomi can now produce computers, as shown in the picture on the following page (the yellow font is what needs to be done to add a new product type), and the factory interface class at the top level also needs to be modified, which is very troublesome.



Abstract factory pattern (expand a product)





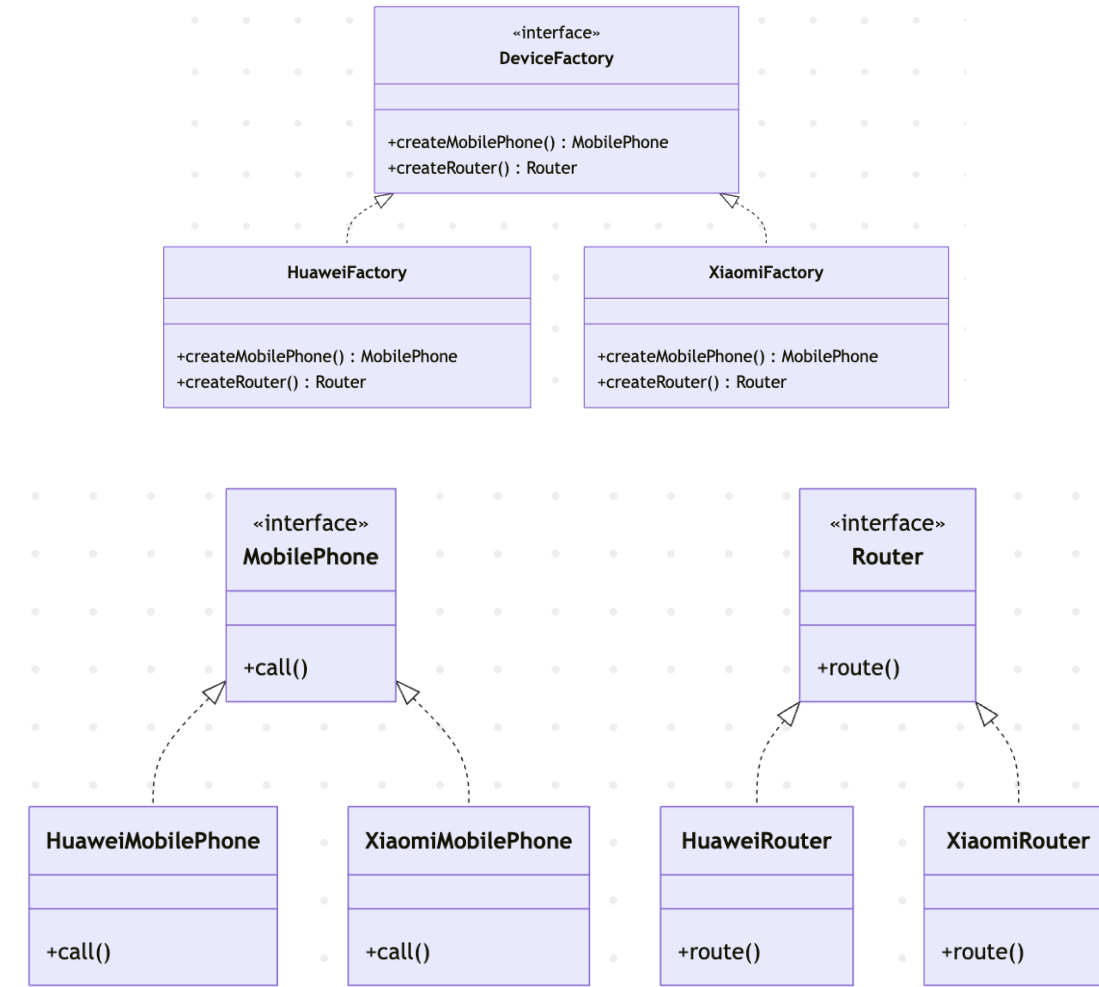
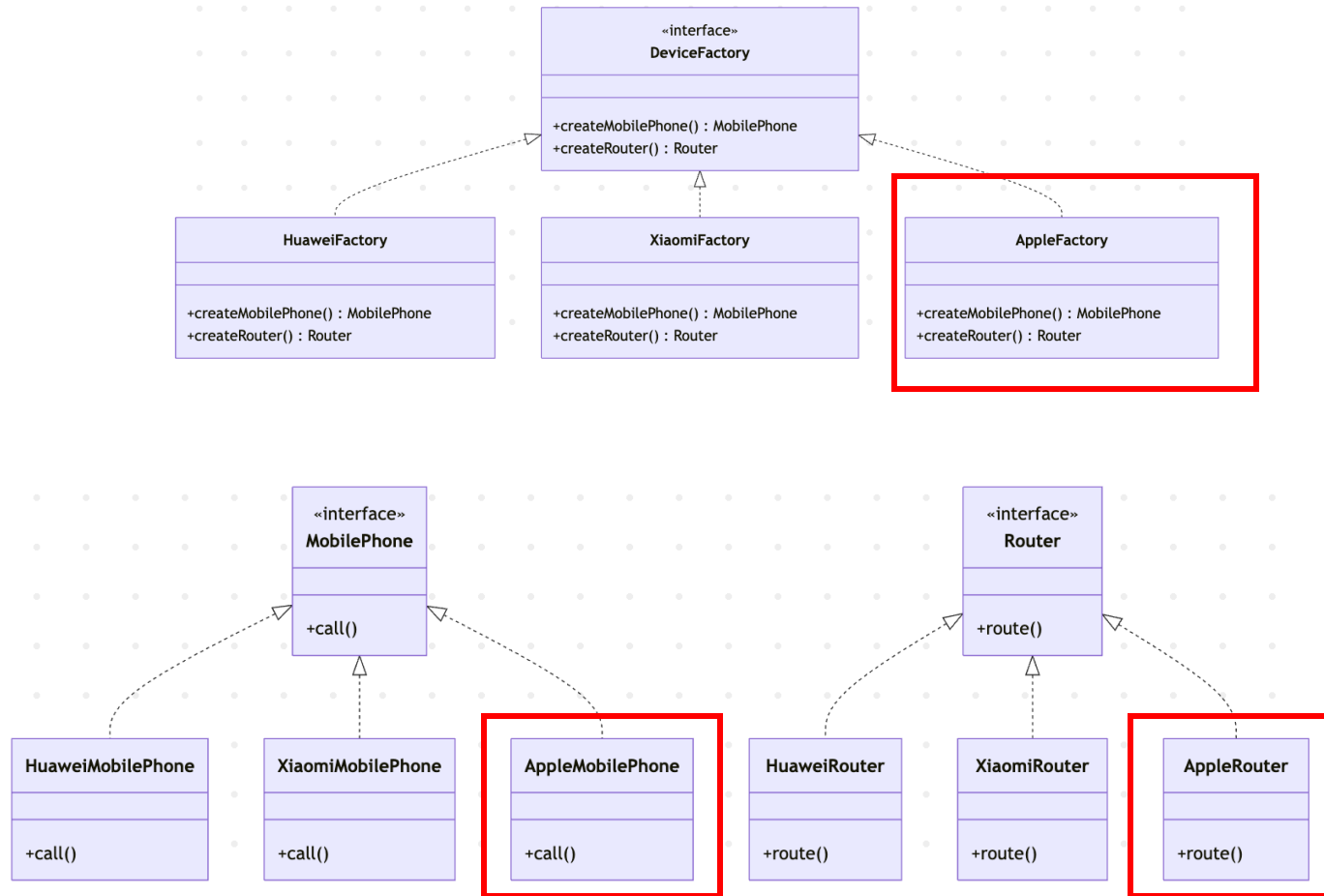
Abstract factory pattern (expand a brand)

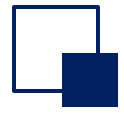
Expand a brand (product family)

- If you add a new brand, that is, add a new manufacturer to produce mobile phones and routers, as shown in the picture on the following page (the yellow font is what you need to do to add a new product family), adding a new product family does not need to modify the original code, which is in line with the OCP principle, which is very convenient.



Abstract factory pattern (expand a brand)





Advantages and disadvantages of the abstract factory pattern

- Advantages

- When multiple objects in a product family are designed to work together, it ensures that the client always uses only objects from the same product family (creating products from one brand together).

- Disadvantages

- It is very difficult to extend the product type, and to add a new product class, you need to modify both the code in the factory abstract class and the code in the specific implementation class.

- It increases the abstraction and difficulty of understanding the system.